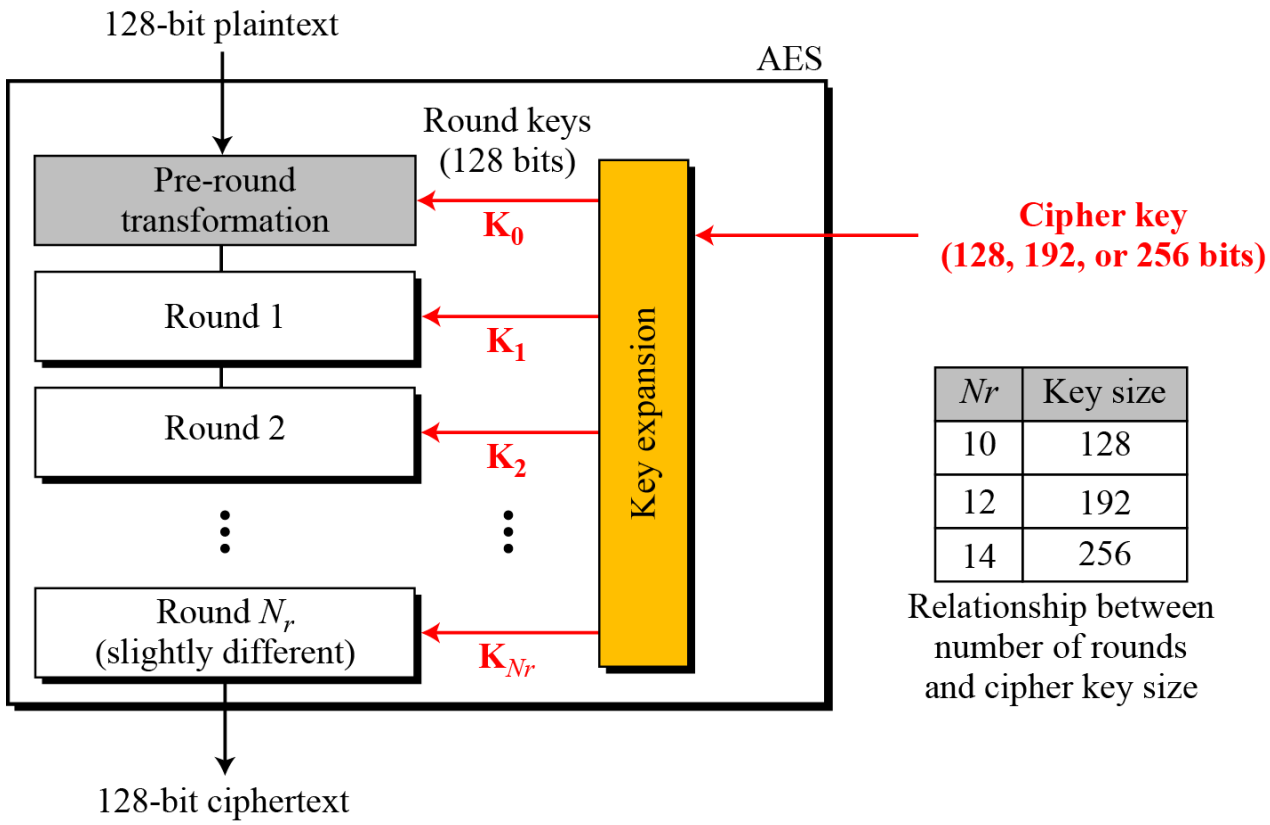
Lecture 4: Public Key Encryption and RSA

ITDS271: Computer and Communication Security

Dolvara Gunatilaka, PhD

Review: AES

- Symmetric block cipher (**128-bit block**)
- Key size: **128/192/256 bits**
- Based on ***Substitution-Permutation Network (SPN)***
- Iterative process → **round**
  - **Round 1- (N-1):** SubBytes, ShiftRows, MixColumns, AddRoundKey
  - **Round N:** SubBytes, ShiftRows, AddRoundKey



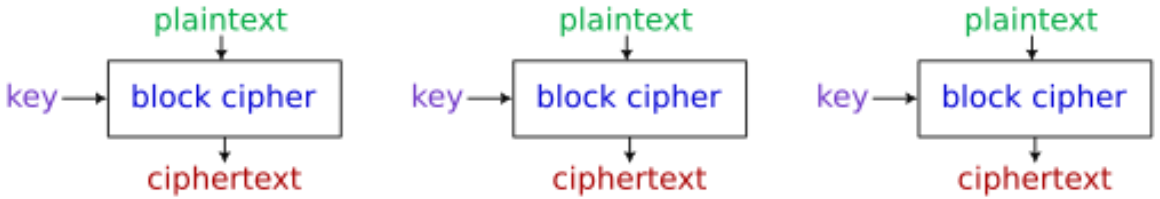
Nr	Key size
10	128
12	192
14	256

Relationship between number of rounds and cipher key size

Review: Mode of Operations

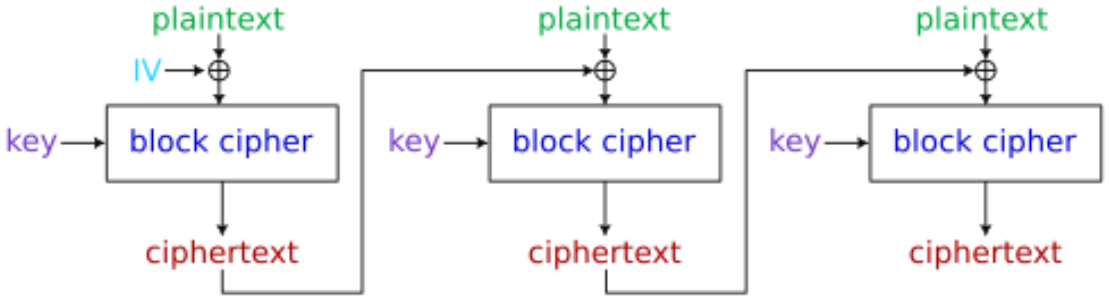
- Handle data that is longer than the block size

Electronic codebook (ECB)



- Simple to implement.
- Allows parallel encryption of blocks (fast)
- Identical plaintext blocks → identical ciphertext blocks.
- Leak patterns

Cipher block chaining (CBC)

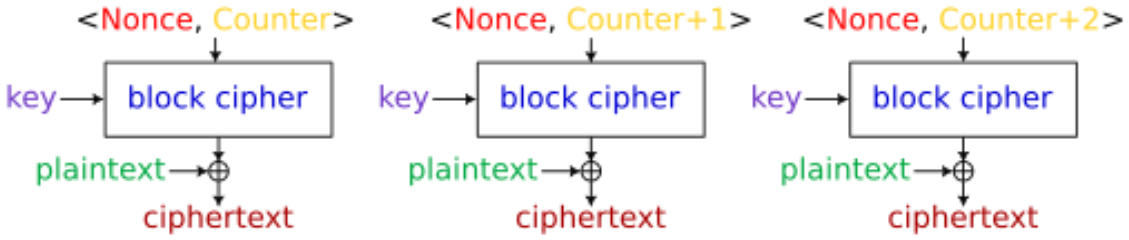


- Chaining removes patterns → more secure than ECB
- Require sequential encryption
- Needs a random IV (must be unique and unpredictable).
- A single-bit error in ciphertext affects two blocks on decryption.

Review: Mode of Operations

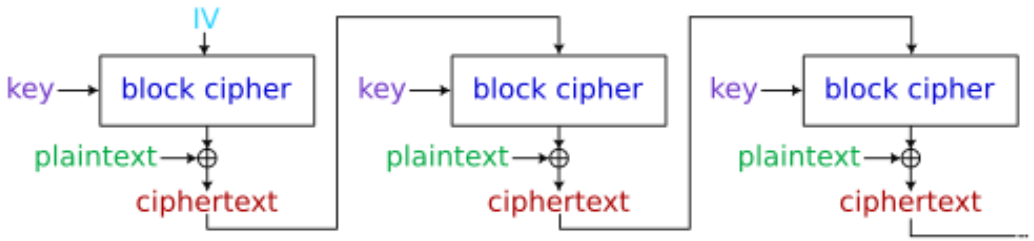
- Stream mode of opertaions
  - Use block ciphers to generate keystream
  - Ciphertext = plaintext XOR keystream

Counter (CTR)



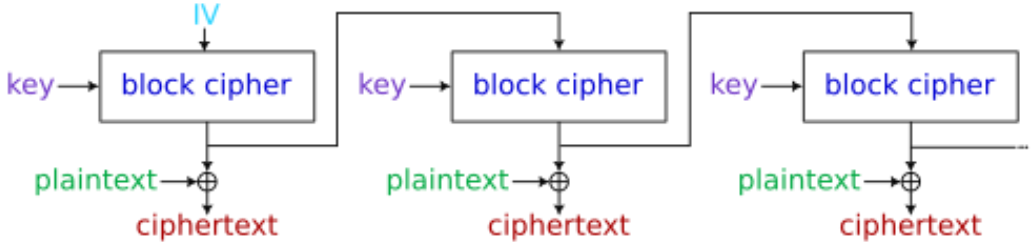
- Fully parallelizable (both encryption & decryption).
- Precomputation of keystream is possible.
- No error propagation
- High efficiency

Cipher feedback (CFB)



- Error propagation
- Sequential encryption

Output feedback (OFB)



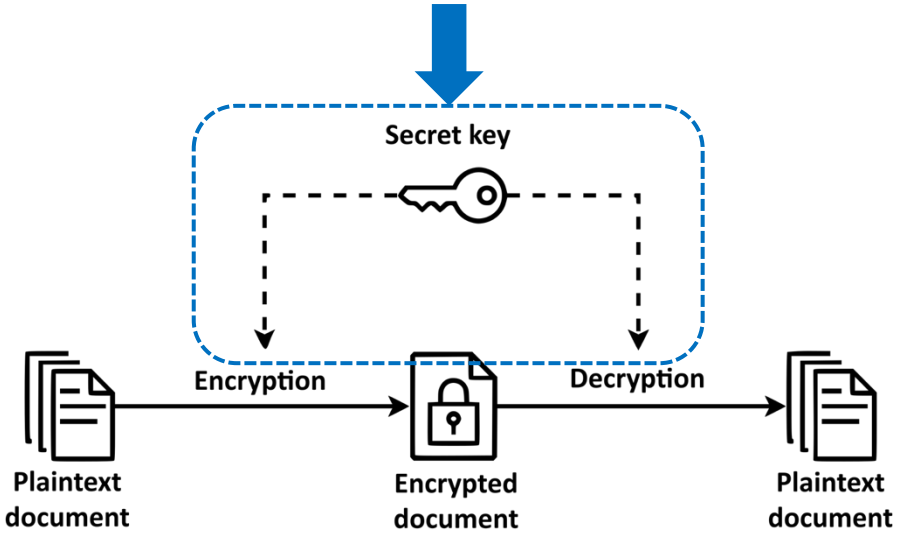
- No error propagation
- Precomputation of keystream is possible.

Topics

- **Public-Key Encryption (PKE)**
- Fundamental Mathematic
- RSA Algorithm
- Key Exchange Algorithm

Drawbacks of Symmetric Key Encryption

- Traditionally, symmetric-key encryption uses <u>one</u> key **shared** by both sender and receiver
- If the key is disclosed, **<u>communications using that key are all compromised</u>**
  - No way to protect sender from the receiver forging a message and claiming that it was sent by the sender.
- ***Key distribution*** (key sharing) is also a problem!



Asymmetric Key Encryption

- Also called **Public-key Encryption (PKE)**
- The most significant advancement in the 3000-year history of cryptography!
- Use clever application of number theoretic concepts to create an encryption function that is easy to calculate but much harder to reverse without sufficient information
- Use as complements rather than replacing symmetric-key encryption

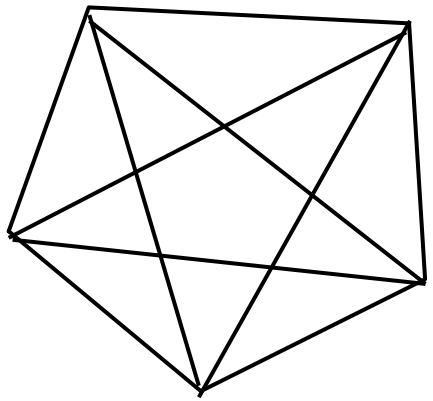
Why PKE?

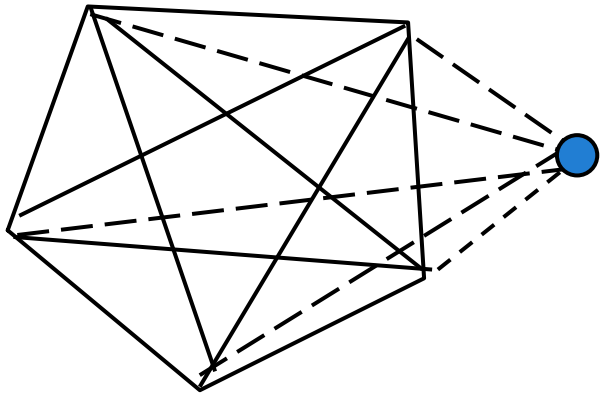
Developed to address 2 main issues:

- **Key distribution:**
  - How to *securely deliver the key* without having to trust a key distribution center (KDC) with your key.
- **Authentication + Non-Repudiation:**
  - How to verify that a message comes intact (without modification) *from the claimed sender*

Key Maintenance Problem

- For conventional system, there must be a common key between any pair of users.
- Therefore, **a lot of keys** must be generated and maintained.
- **Symmetric: *n(n-1)/2* keys** for *n* users
- **Asymmetric (PKE)**: each user needs to maintain only 2 keys.
  - ***Public key*** and ***private key***
  - ***2n* keys** are enough for n users.

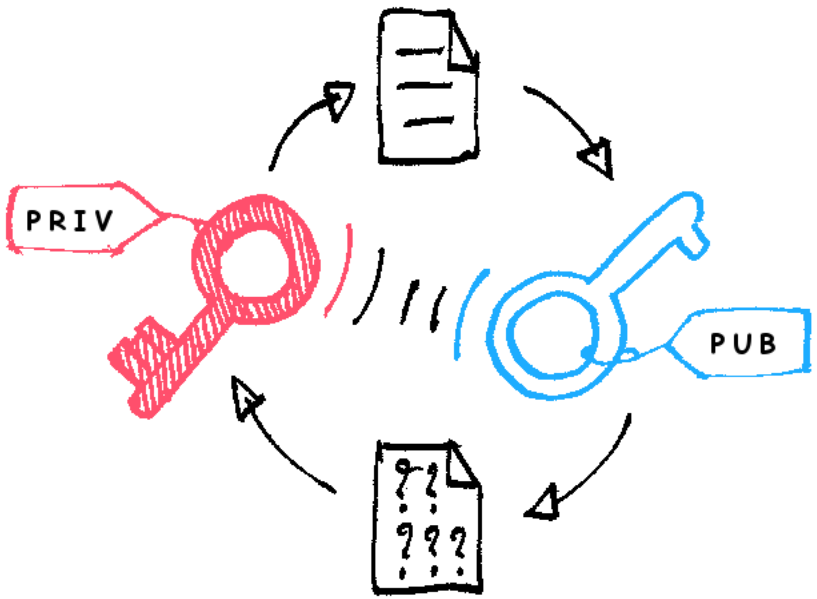




Public Key Encryption

PKE involves 2 keys:

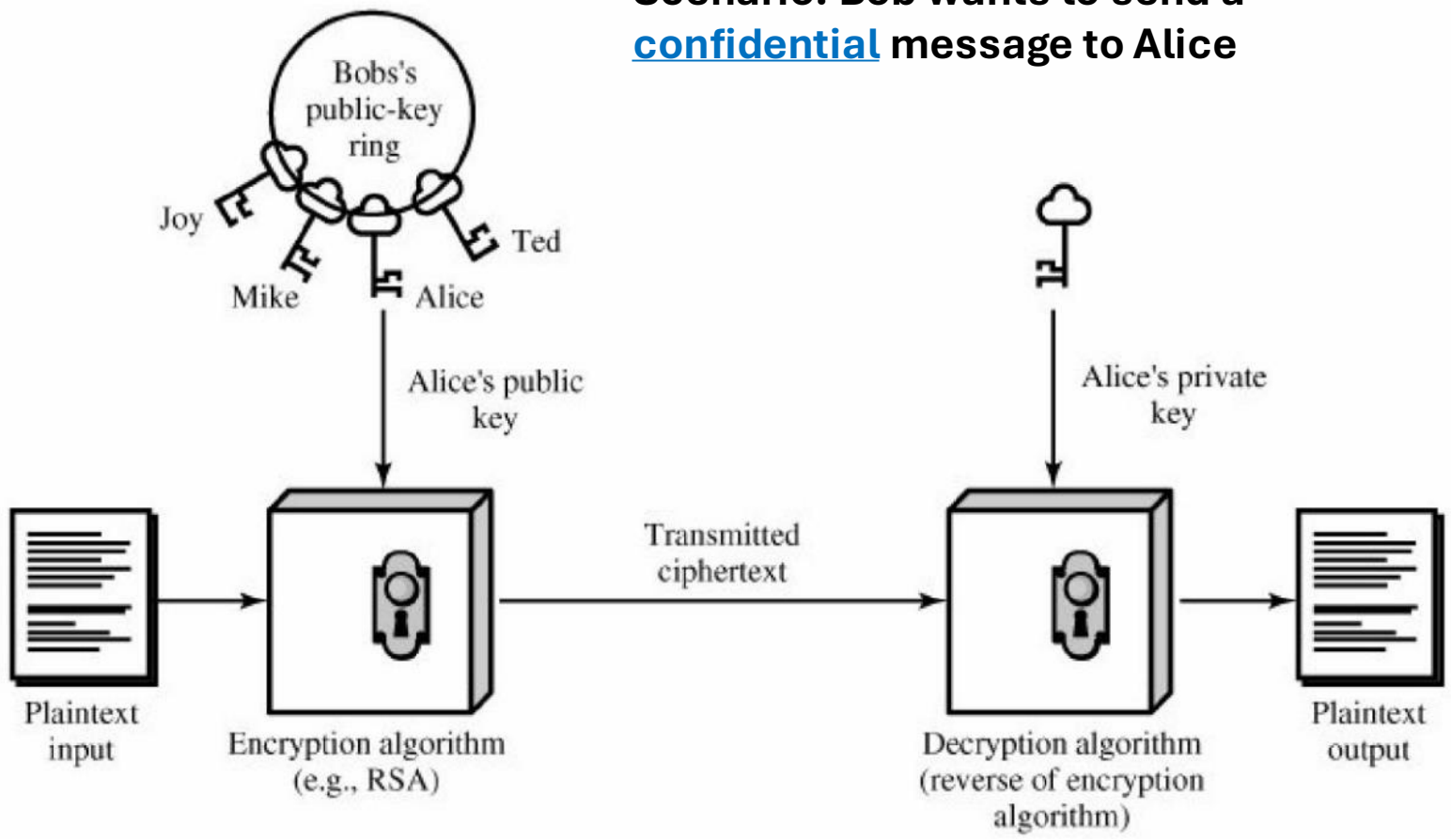
- ***Public-key:***
  - Known by anyone (public)
  - Used for encrypt messages and verify signatures
- ***Private-key:***
  - Known only to the recipient
  - Used to decrypt the message and sign (create) signatures



- It is *asymmetric* because:
  - Those who encrypt the messages or verify the signatures cannot decrypt messages or create signatures.

Public Key Encryption: Data Encryption

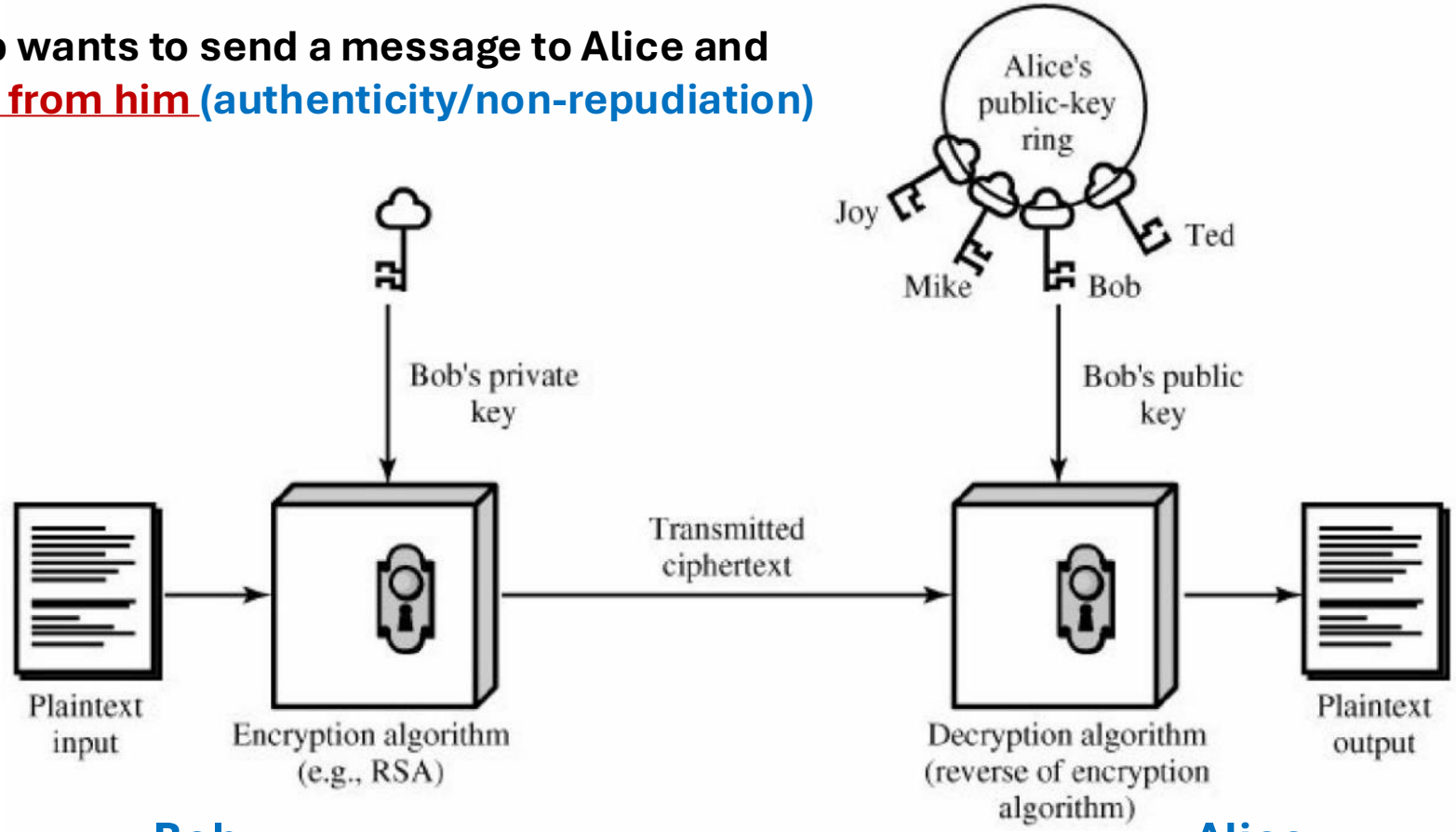
Scenario: Bob wants to send a confidential message to Alice



Bob **Alice**

Public Key Encryption: Digital Signature

Scenario: Bob wants to send a message to Alice and prove it came from him (authenticity/non-repudiation)



Bob **Alice**

Summary of PKE Characteristics

- Solve the **key distribution problem**; no need to transmit a secret key over the network while the public key can be freely distributed
- Highly ***scalable***; suitable for large-scale systems
- Support ***digital signature***
- Based on hard ***mathematical problem***
  - Computationally infeasible to derive the private key from the public key
- Computationally expensive; slower than symmetric key encryption
- Like symmetric-key encryption schemes, brute force (exhaustive) search is always theoretically possible

Topics

- Public-Key Encryption (PKE)
- **Fundamental Mathematic**
- RSA Algorithm
- Key Exchange Algorithm

Prime Number

- Prime number = natural number greater than **1** that
  - Has **exactly two distinct positive divisors; 1 and itself**
  - **1 is not prime** because it has only 1 divisor
- Cannot be written as a product of other numbers
- Examples of Prime Numbers: 2, 3, 5, 7, 11, 13, 17, 19, 23, …

Modular Arithmetic

- For any integers, a and n:
  - ***a* mod *n* = *r*** ***r* = remainder when *a* is divided by *n***
  - ***a* = *q**n* + *r*** where $0 \leq r < n$
- All integers can be written: $a = q*n + r$ where $0 < r < n$
  - e.g., 100 mod 11 = 1 → 100 = 9*11 + 1
- Two integers ***a*** and ***b*** are said to be ***congruent modulo n***, if they produce the same remainder when divided by *n*, written as ***a ≡ b mod n***
  - 17 ≡ 5 mod 12
  - -12 ≡ -5 ≡ 2 ≡ 9 ≡ … mod 7

Multiplicative Inverse in Moduluar Arithmetic

- ***x*** is a multiplicative inverse of ***a mod n*** if ***(x*a) ≡ 1 mod n***
- Multiplicative inverse of ***a mod n*** exists if and only if ***a*** and ***n*** are ***coprime***
  - No common divisors apart from 1
  - ***gcd(a,n) = 1***
- For Example:
  - **x = 37** is the inverse of **13 mod 60**
  - Because 13*37 mod 60 = 1

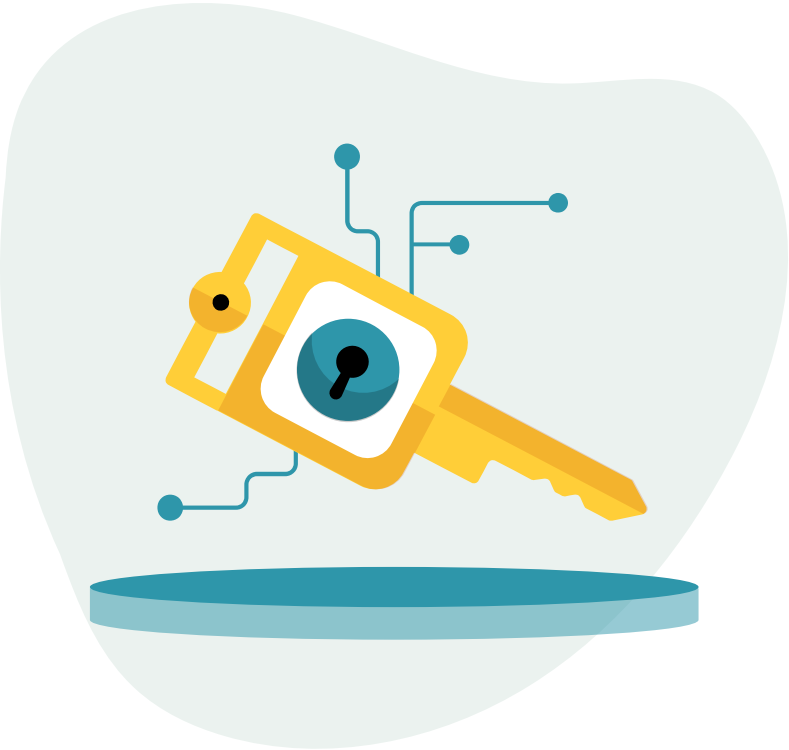
Is 7 a multiplicative inverse of 9 mod 31 ?

Topics

- Public-Key Encryption (PKE)
- Fundamental Mathematic
- **RSA Algorithm**
- Key Exchange Algorithm

RSA

- Invented by **R**ivest, **S**hamir and **A**dleman of MIT in 1978
- Best known and widely used **public-key** encryption scheme
- Security relies on the cost of factoring large prime numbers
  - Factorization problem takes $O(e^{(\log n)(\log \log n)})$ operations → **hard**
- Used for TLS/SSL key exchange, digital signature, code signing



RSA Key Generation

Each user generates a pair of public ***PU = {e,n}*** and private ***PR = {d,n}*** *keys*

1. Randomly choose two large prime number: p and q
2. Compute n (modulus) $n = p \times q$
3. Calculate Euler's Totient function $\phi(n)=(p-1)\times(q-1)$
4. Randomly select an integer **e** (public) satisfying the following conditions:
   - $1 < e < \phi(n)$ and $\gcd(e, \phi(n))=1$
5. Compute **d** (private) where $de \equiv 1 (\mathrm{mod}\, \phi(n))$
   - **d** is the modular multiplicative inverse of **e modulo ϕ(n)**
   - Can be computed using the Extended Euclidean algorithm

Publish the public-key:
PU = {e, n}

Keep the private-key:
PR = {d, n}

RSA Encryption and Decryption

- To <u>encrypt</u> a **message M**, the sender:
  - Obtains the ***public-key of <u>recipient</u>*** — $PU = \{e, n\}$
  - Computes: — $C = M^e \bmod n$, where $0 \leq M < n$
- To <u>decrypt</u> the **ciphertext C**, the recipient:
  - Uses their ***private-key*** — $PR = \{d, n\}$
  - Computes: — $M = C^d \bmod n$
- Note that the message *M* must be smaller than the modulus *n* (separate *M* into blocks if needed)

Example: RSA Key Generation

1)	Select primes:	$p = 17$, $q = 11$
2)	Compute	$n = p * q = 17 * 11 = 187$
3)	Compute	$\emptyset(n) = (p-1)(q-1) = 16 * 10 = 160$
4)	Select e:	$\boldsymbol{gcd(e, 160) = 1}$, **choose** $\boldsymbol{e = 7}$
5)	Determine d:	$\boldsymbol{de \equiv 1 (mod\ \phi(n)),\ d < 160,\ d = 23}$ (since $23 * 7 = 161 = 1 \bmod 160$)
6)	Publish public-key	$\boldsymbol{PU = \{7, 187\}}$
7)	Keep the private-key	$\boldsymbol{PR = \{23, 187\}}$

Example: RSA Encryption and Decryption

Given a message $M = 88$

- **Encryption (PU = {7,187}):**
  - $C = 88^7 \bmod 187 \quad = 11$
- **Decryption (PR = {23, 187}):**
  - $M' = 11^{23} \bmod 187 = 88$

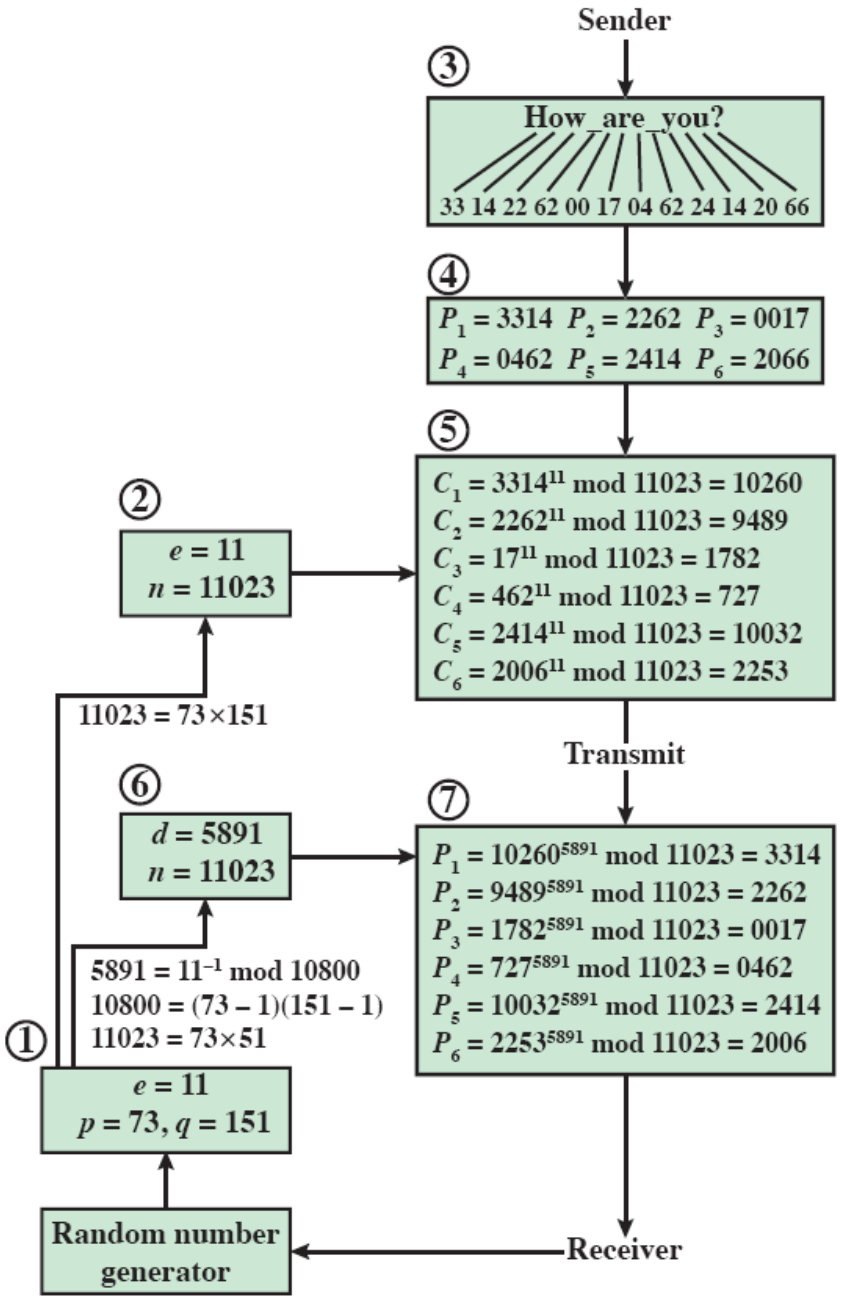
Example: RSA Cryptosystem

$PU = \{e, n\}$

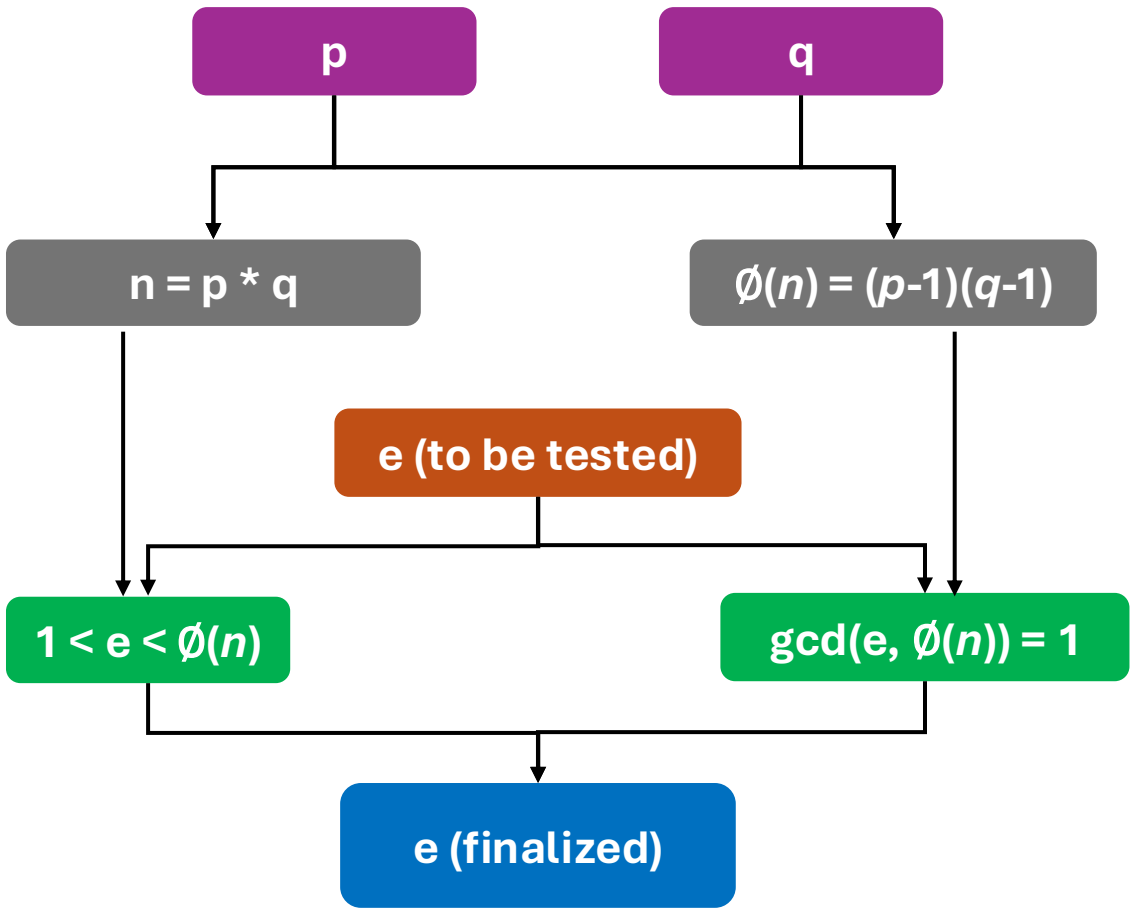
$C = M^e \bmod n$, where $0 \le M < n$

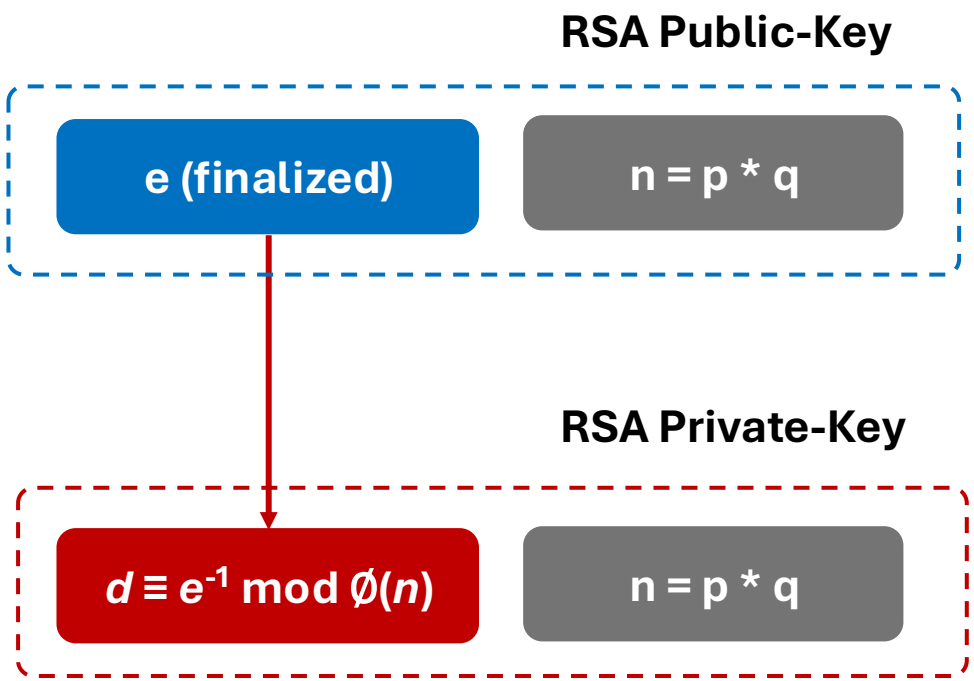
$PR = \{d, n\}$

$M = C^d \bmod n$



Summary of RSA Algorithm





Security of RSA

- Computational difficulty of factoring large prime **n**
  - If an attacker can factor n into p and q, they can calculate ϕ(n) and then d
- Size of the key: RSA security increases with key size
  - Minimum recommended **2048-bit key**
- Attacks on RSA:
  - **Brute-force Attack:** practically impossible due to the key size
  - **Factoring Attacks:** attempt to find p and q.
  - **Timing Attacks:** analyze the time taken for decryption operations.
  - **Chosen Ciphertext Attacks (CCA):** Exploit properties of RSA if not properly padded (e.g., using OAEP).

Security of RSA: Factoring Problem

- **RSA-250 (829 bits)** is the largest RSA challenge number factored so far (2020)

RSA-250 Status: Factored (Feb 2020)

829 bits / 250 digit (decimal):

2140324650240744961264423072839333563008
6147151447550177977549208814180234471
4013664334551909580467961099285187247091
4587687396261921557363047454770520805
1190564931066876915900197594056934574522
3058932597669747168173806936489469
9871578494975937497937

Approximately **2700 CPU core-years**, using a 2.1 GHz Intel Xeon Gold 6130 CPU

Factors:

6413528947707158027879019017057738908482501
4742943447208116859632024532344630238623598752668347708737661925585694639798853367

and

3337202759497815655622601060535511422794076034476755466678452098702384172921003708025744867329688187756571898625803693206271 1

Other Asymmetric Cryptosystems

- **Elliptic Curve Cryptography (ECC)**
  - Same security level as RSA, but requires smaller key size making it more efficient
  - Instead of prime factorization problem, security of ECC relies on solving discrete logarithm problem
  - Widely used in modern protocols, e.g., TLS1.3, Elliptic Curve Diffie-Hellman (ECDH) key exchange, ECDSA for digital signature
- **Elgamal Encryption**
  - Cryptographic algorithm based on the Diffie–Hellman key exchange
  - Slower than RSA for encryption and faster for decryption
  - Less common, used in PGP, theoretical interest

Topics

- Public-Key Encryption (PKE)
- Fundamental Mathematic
- RSA Algorithm
- **Key Exchange Algorithm**

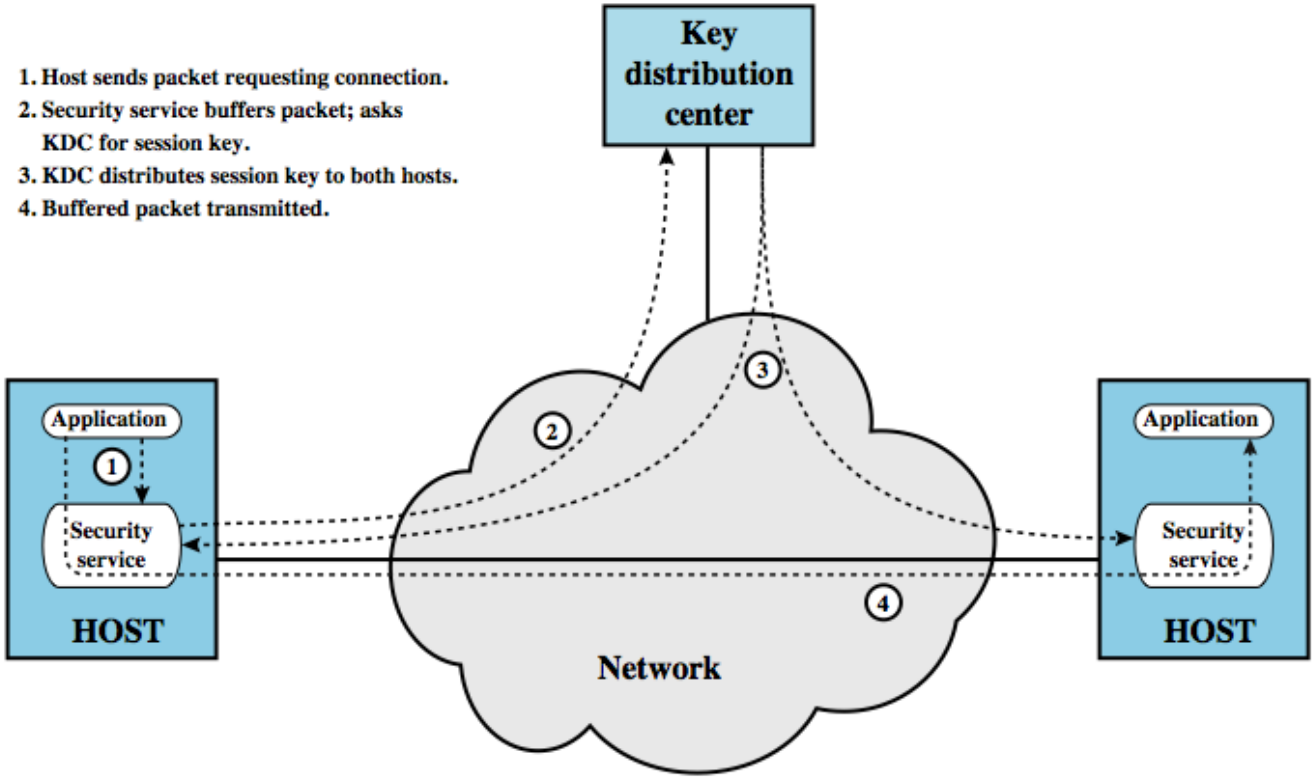
Applications of PKE

- PKE is <u>much slower</u> than **symmetric-key encryption**
- Applications of PKE in practice
  1) **Key Exchange: key transport vs key agreement**
  2) Message Authentication/Message Integrity
  3) Digital Signatures
  4) Digital Certificates

How to securely <u>exchange/establish the secret key</u> between two parties ?

Traditional Key Exchange: KDC

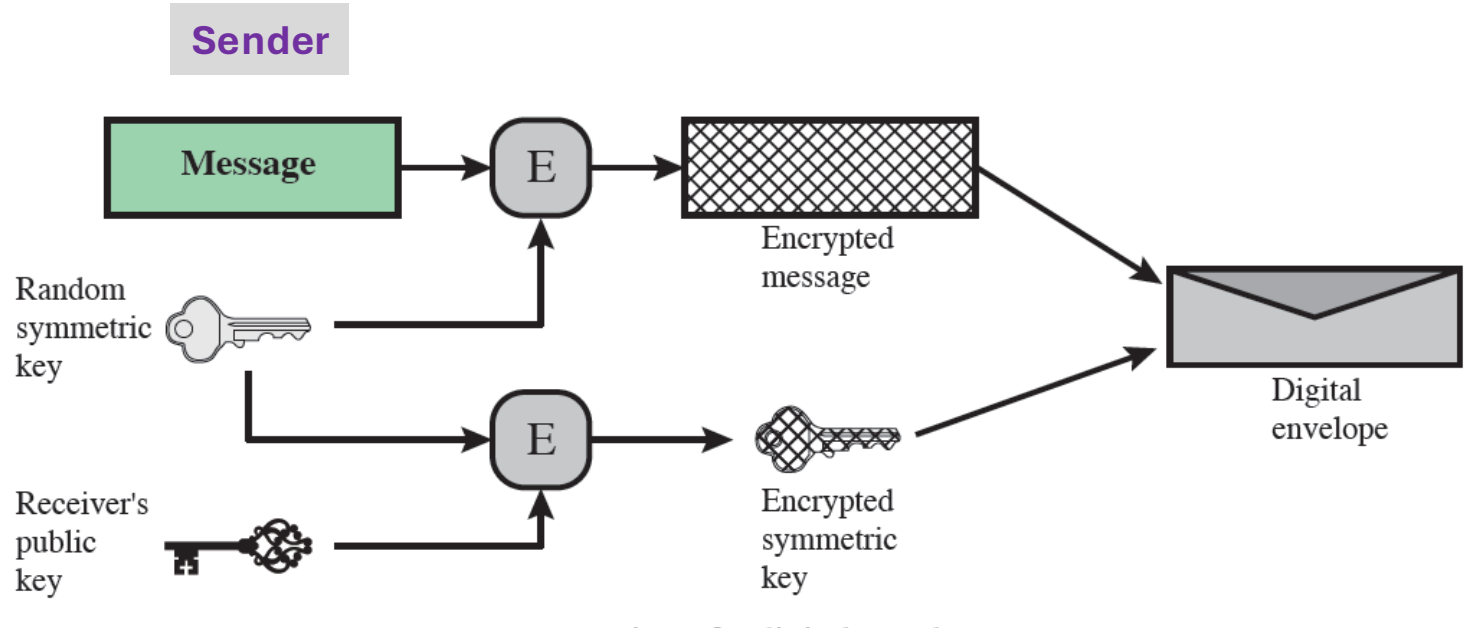
- Traditional way of exchanging key
- Using ***Key Distribution Center (KDC)***
- Rely on <u>trusted third party</u>
- Distributed session key is encrypted with each host pre-shared secret key
- **<u>Limitations:</u>** require every party to have pre-shared secret key with the central KDC



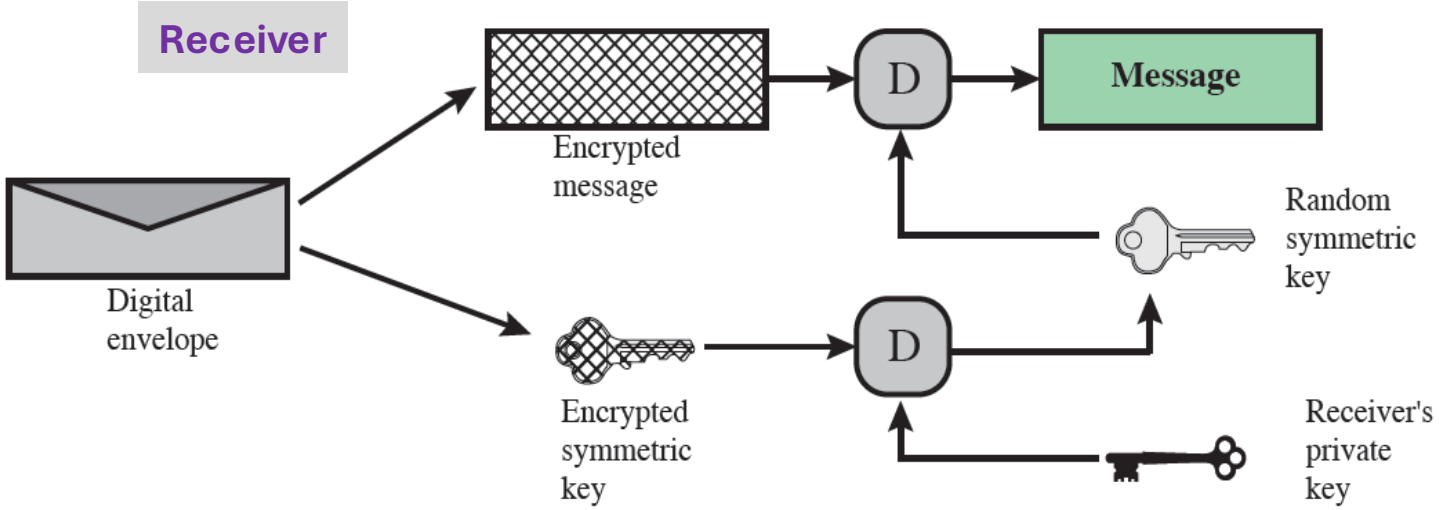
Key Exchange: Key Transport

PKE for Key Transport:

- Sender securely share **symmetric key** (for data encryption) with receiver
- Sender uses **receiver's public key** to <u>encrypt</u> **symmetric key**
- Receiver uses its own **private key** to <u>decrypt</u> **symmetric key**



Creation of Digital Envelope



Opening of Digital Envelope

Key Exchnage: Key Transport Protocol

Key Transport: one party **generates the key** and **securely sends** it to the other

Encryption (by sender) :

1. Choose random x
2. Encrypt $y = \mathrm{RSA}(e, x) = x^e \bmod n$
   (Asymmetric-key Encryption)
3. Compute session key: $k = H(x)$
   (H(?): Hash function)
4. $c = \mathrm{E}(k, m)$
   (Symmetric-key Encryption)
5. Output ciphertext (y, c)

Decryption (by receiver) :

1. $x = \mathrm{RSA}(d, y) = y^d \bmod n$
   (Asymmetric-key Decryption)
2. Compute session key : $k' = \mathrm{H}(x')$
3. $m = \mathrm{D}(k, c)$
   (Symmetric-key Decryption)

Note: Can replace RSA with ECC for key transport

Key Exchange: D-H Algorithm

- Known as **"Diffie-Hellman Key Exchange Algorithm"**
- Proposed by **Diffie and Hellman in 1976** along with the exposition of PKE concept
  - Cannot be used to exchange arbitrary message
  - On the other hand, it can **help establish a common key**
- D-H is a ***key agreement*** protocol for public exchange of a secret key
  - Both parties **contribute information** to derive the **same shared key**, without explicitly sending the key itself.
- Used in several commercial products
  - Usually integrated into larger protocols, e.g., TLS/SSL, SSH, IPSec

Primitive Root

- Let **p** be a **prime number**.
- An integer **g** is called a **primitive root modulo p** if: $g^1, g^2, g^3, \ldots, g^{\{p-1\}}$ ***(mod p)***
- produce <u>all non-zero integers</u> **modulo p** exactly once: **{1, 2, 3, .. p-1}**
- {1, 2, 3, .. p-1} are <u>coprime</u> to **p**
- ***Is 2 a primitive root of mod 7?***

<u>Example</u>: **p = 7,**

then the number **coprime to 7** is **{1,2,3,4,5,6}**

Let's test **g = 3**

- 3^1 mod 7 = 3
- 3^2 mod 7 = 2
- 3^3 mod 7 = 6
- 3^4 mod 7 = 4
- 3^5 mod 7 = 5
- 3^6 mod 7 = 1

The powers of 3 generate all numbers from 1 to 6 → so **3 is a primitive root mod 7**

How D-H Algorithm Works (1)

- Public Parameters (agreed upon by both parties)
  - **p**: a large prime number
  - **g:** a primitive root modulo p
- Each party generates their own secret key
  - Chooses a secret integer (private): **x,** where **x < p**
  - Compute a public key : $y = g^x \bmod p$
- Two parties: A and B
  - A: Keep secret integer x_A and publish public key y_A
  - B: Keep secret integer x_B and publish public key y_B

How D-H Algorithm Works (2)

- A <u>shared session key</u> for A and B is denoted as K_{AB}

$$
\begin{aligned}
K_{AB} &= Y_A^{X_B} \bmod p \\
&= Y_B^{X_A} \bmod p \\
&= g^{X_A X_B} \bmod p
\end{aligned}
$$

- K_{AB} is used as a session key in symmetric-key encryption scheme between A and B
- If A and B subsequently communicate, they will have the <u>same key as before</u> (unless they choose new public-keys)

How D-H Algorithm Works?

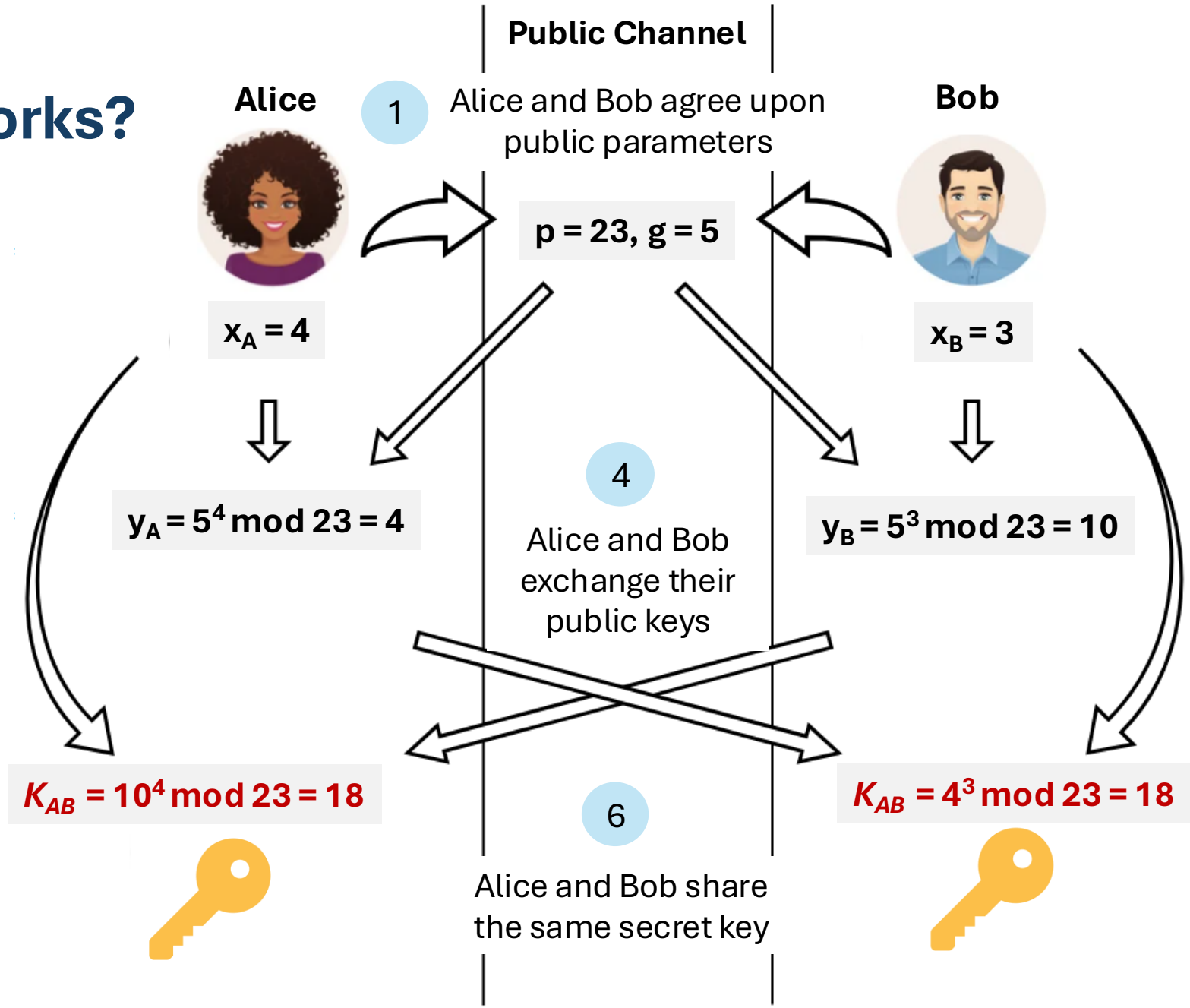


Image: https://en.wikipedia.org/wiki/Diffie%E2%80%93Hellman_key_exchange

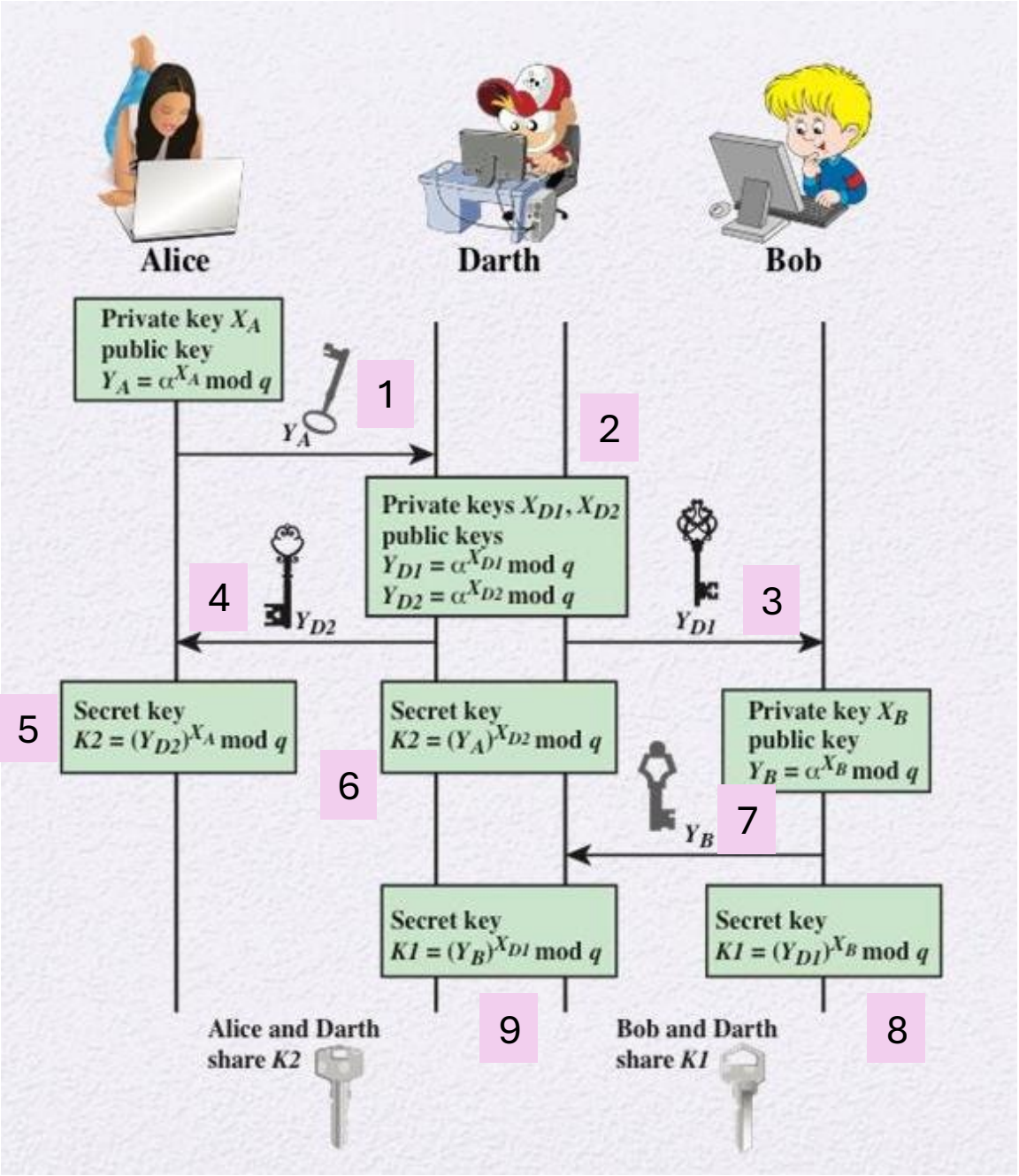
Security of D-H Algorithm

- An eavesdropper, Eve, knows the public parameters **p** and **g**, as well as the public values y_A and y_B.
  - To find the shared secret, Eve would need to find either Alice's private key x_A from $y_A = g^{X_A} \bmod p$ <u>or</u> Bob's private key x_B from $y_B = g^{X_B} \bmod p$
  - This is known as the **discrete logarithm problem**; for a large prime p, it is computationally infeasible to solve
- Diffie-Hellman is considered secure against <u>passive</u> eavesdropping
- Security depends on <u>proper parameters</u>, i.e., **p** should be large (at least 2048 bits), **g** must be primitive root, and **x** must be truly random and relatively large
- But, vulnerable to a ***Man-in-the-Middle (MITM)*** attack → **no authentication**

Security of D-H Algorithm: MITM

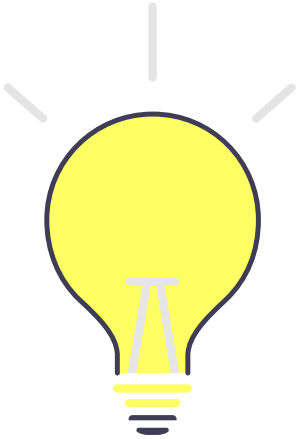
Darth can intercept all messages between Alice and Bob.

- Alice and Bob computes and sends their public key $\mathbf{Y_A}$ and $\mathbf{Y_B}$
- Darth intercepts $\mathbf{Y_A}$ and $\mathbf{Y_B}$
- Darth creates two public keys and sends one to Bob $\mathbf{Y_{D1}}$ and one to Alice $\mathbf{Y_{D1}}$
- Two different shared keys are created
  - **K1**:between Bob and Darth
  - **K2**: between Alice and Darth



Summary

- **Publick key encryption (PKE)** or **Asymmetric key encryption**
  - Require a pair of **private** and **public** keys
  - Depend on hard math problem → difficult to break
  - Slower and more complex than symmetric key encryption
- Based on number theory: prime and co-prime and modular arithmetic
- **RSA** algorithm is an example of PKE
- PKE is used for <u>key exchange</u>
  - **Key transport**: use RSA, ECC, etc., to transmit shared secret key
  - **Key agreement: Diffie-Hellman algorithm**



มหาวิทยาลัยมหิดล
Mahidol University

ห้ามมิให้นักศึกษาทำซ้ำ ดัดแปลง หรือใช้ประโยชน์จากงานอันมีลิขสิทธิ์ปรากฏอยู่ในระบบการศึกษาอิเล็กทรอนิกส์ (E-Learning) ของมหาวิทยาลัย ไม่ว่าจะทั้งหมดหรือบางส่วนโดยไม่ได้รับอนุญาตจากมหาวิทยาลัย นอกเหนือจากการศึกษาส่วนบุคคล ทั้งนี้ การทำซ้ำ ดัดแปลง หรือเผยแพร่ต่อสาธารณชนซึ่งงานอันมีลิขสิทธิ์ **จะมีโทษปรับตั้งแต่ 20,000 บาท ถึง 200,000 บาท และหากเป็นการกระทำเพื่อการค้า จะมีโทษจำคุกตั้งแต่ 6 เดือน ถึง 10 ปี หรือปรับตั้งแต่ 100,000 บาท ถึง 800,000 บาท หรือทั้งจำทั้งปรับ**

Unless allowed by Mahidol University, the User shall not copy, modify, or exploit in part or in whole of the copyrighted materials on the Platform, other than for your own individual study. The copy, modify or communication to public of the copyrighted materials shall be inflicted with a fine from 20,000 Baht up to 200,000 Baht. If the offence is committed with the commercial purpose, the offender shall be inflicted with imprisonment for a term from 6 months up to 10 years or a fine from 100,000 Baht up to 800,000 Baht or both imprisonment and fine.